

WebMacro

Share SQL macros through a trusted URL.



Install & load

```
INSTALL webmacro FROM community;
LOAD webmacro;
```

01 At a glance

Load a DuckDB macro definition from an HTTP or HTTPS endpoint and call it in ordinary SQL. Use reviewed, versioned SQL files to distribute reusable scalar expressions and table queries across notebooks and applications.

SQL at a URL → Load definition → Call the macro

02 Loading & macro lifecycle

`load_macro_from_url(url)`

Fetch and register a macro; return a status string.

`CREATE MACRO name(args) AS expression`

Define a scalar macro in the hosted SQL file.

`CREATE MACRO name(args) AS TABLE ...`

Define a table macro called from a FROM clause.

`CREATE OR REPLACE MACRO ...`

Let reloading replace a previously registered definition.

`duckdb_functions()`

Inspect registered macros with `function_type = 'macro'`.

`DROP MACRO name`

Remove a loaded macro with DuckDB's ordinary lifecycle command.

03 Keep in mind

The SQL file is executable code

Load only reviewed SQL from a trusted host. The extension's keyword checks are not a sandbox. Use HTTPS and immutable revision URLs when reproducibility matters; a branch URL can change after review.

04 Prepare a reusable definition

```
CREATE OR REPLACE MACRO normalize_tag(s)
AS lower(trim(s));
SELECT normalize_tag(' Farm ') AS tag;
-- Result: farm
```

Test locally, then publish the CREATE statement as `normalize_tag.sql` on your host.

05 Load the hosted SQL

REQUIRES SETUP

```
SELECT load_macro_from_url(
  'https://example.com/normalize_tag.sql'
);
SELECT normalize_tag(' Farm ') AS tag;
```

Replace the placeholder URL with the trusted SQL file prepared above.

06 Inspect and remove a macro

```
SELECT function_name, macro_definition
FROM duckdb_functions()
WHERE function_type = 'macro'
  AND function_name = 'normalize_tag';
DROP MACRO normalize_tag;
```

After creation or loading, it behaves like an ordinary DuckDB macro.

07 In practice

Load dependencies first

Each call fetches its URL again. There is no dependency resolver: load prerequisite macros before dependents, and load required extensions before use. Plain GET is the interface; private hosts need an accessible or presigned URL.

[Full docs & examples ↗](#) [Source ↗](#)